

Roteirize — Especificação Técnica do Projeto

Projeto final da disciplina de Front-end e Back-end App web para criação e gestão de roteiros de viagem.

O nome "**Roteirize**" é apenas uma sugestão — sinta-se à vontade para renomear o projeto. Este documento foi escrito para que um aluno iniciante consiga implementar o sistema do zero, seguindo a ordem sugerida na seção 12.

Sumário

1. Visão geral do projeto
 2. Funcionalidades essenciais
 3. Arquitetura da aplicação
 4. Estrutura de pastas e arquivos
 5. Banco de dados (MySQL)
 6. Fluxo de autenticação
 7. Telas do sistema (com textos)
 8. Especificação da API (back-end PHP)
 9. Organização do front-end (CSS e JS)
 10. Responsividade
 11. Segurança — boas práticas mínimas
 12. Roteiro sugerido de implementação
 13. Funcionalidades extras (opcional)
 14. Checklist de testes manuais
-

1. Visão geral do projeto

O **Roteirize** é uma aplicação web que permite que um usuário crie uma conta, faça login e organize suas viagens em **roteiros**. Cada roteiro tem um destino, uma data de início e fim, e uma lista de **atividades** (passeios, transportes, hospedagens, refeições etc.) distribuídas pelos dias da viagem.

Objetivo didático: praticar a construção de uma aplicação completa, dividida em:

- **Front-end** (HTML semântico + CSS responsivo + JavaScript puro) rodando no navegador;
- **Back-end** (PHP) expondo uma pequena API que recebe e devolve dados em JSON;
- **Banco de dados** (MySQL) persistindo usuários, roteiros e atividades.

Escopo mínimo (MVP) — o que o projeto **precisa** ter:

- Cadastro e login de usuários;
- Criar, listar, editar e excluir roteiros;
- Criar, listar, editar e excluir atividades dentro de um roteiro;
- Visualizar as atividades organizadas por dia;
- Layout responsivo (desktop, tablet, celular).

Tudo que passar disso (upload de imagens, mapas, exportar PDF etc.) é tratado como **extra opcional** na seção 13, para quem quiser ir além do mínimo.

2. Funcionalidades essenciais

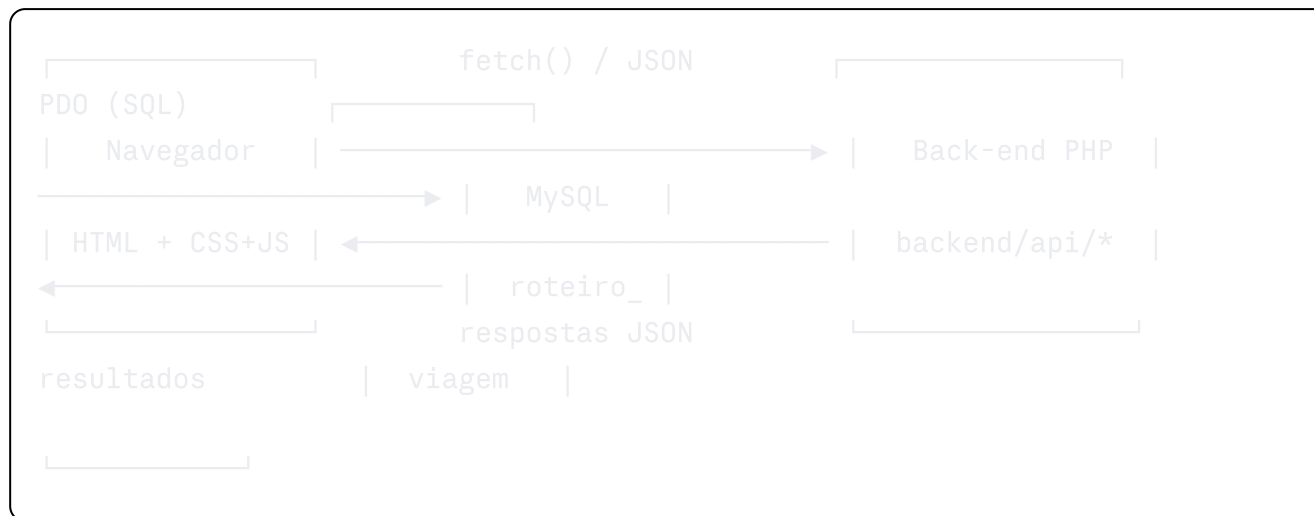
Código	Funcionalidade	Descrição
RF01	Cadastro de	Visitante cria uma conta com nome, e-mail e senha

Código	Funcionalidade	Descrição
	usuário	
RF02	Login	Usuário autentica com e-mail e senha
RF03	Logout	Usuário encerra a sessão
RF04	Criar roteiro	Usuário logado cria um novo roteiro de viagem
RF05	Listar roteiros	Usuário vê todos os roteiros que criou
RF06	Editar roteiro	Usuário altera título, destino, datas ou descrição
RF07	Excluir roteiro	Usuário remove um roteiro (e todas as suas atividades)
RF08	Criar atividade	Usuário adiciona uma atividade a um roteiro, com data e horário
RF09	Listar atividades	Atividades aparecem organizadas por dia dentro do roteiro
RF10	Editar atividade	Usuário altera dados de uma atividade
RF11	Excluir atividade	Usuário remove uma atividade
RF12	Editar perfil	Usuário altera nome e/ou senha
RF13	Proteção de rotas	Um usuário não pode ver/editar roteiros de outra pessoa; páginas internas exigem login

Requisitos não funcionais: interface responsiva, mensagens de erro claras, senhas nunca guardadas em texto puro, proteção contra SQL Injection.

3. Arquitetura da aplicação

O front-end é **estático** (arquivos `.html`, `.css`, `.js`) e conversa com o back-end **exclusivamente via** `fetch()`, trocando dados em **JSON**. O PHP não imprime HTML — ele funciona como uma pequena **API**. Essa separação deixa bem claro o que é "trabalho de front-end" e o que é "trabalho de back-end".



Autenticação: sessão nativa do PHP (`$_SESSION`). Ao fazer login com sucesso, o back-end grava o `id` do usuário na sessão; um cookie de sessão é enviado ao navegador automaticamente e reenviado a cada requisição.

Dica prática: para evitar problemas de CORS durante o desenvolvimento, coloque as pastas `frontend/` e `backend/` dentro do mesmo servidor (ex.: `htdocs` do XAMPP), acessando tudo por `http://localhost/roteirize/frontend/...`. Se preferir rodar o front-end em outra porta (ex.: extensão "Live Server"), veja a nota de CORS na seção 6.

4. Estrutura de pastas e arquivos

```
roteirize/
├── backend/
│   ├── config/
│   │   └── database.php          # conexão PDO com o MySQL
│   ├── includes/
│   │   ├── auth_check.php       # verifica se há usuário logado na
sessão
│   │   ├── cors.php             # cabeçalhos de CORS (uso opcional em
dev)
│   │   └── funcoes.php          # funções auxiliares (validações,
respostas JSON)
│   ├── api/
│   │   ├── auth/
│   │   │   ├── registrar.php
│   │   │   ├── login.php
│   │   │   ├── logout.php
│   │   │   └── usuario_logado.php
│   │   ├── roteiros/
│   │   │   ├── criar.php
│   │   │   ├── listar.php
│   │   │   ├── detalhes.php
│   │   │   ├── editar.php
│   │   │   └── excluir.php
│   │   ├── atividades/
│   │   │   ├── criar.php
│   │   │   ├── listar.php
│   │   │   ├── editar.php
│   │   │   └── excluir.php
│   │   └── perfil/
│   │       └── atualizar.php
```

```
|   └─ database/
|       └─ schema.sql           # script de criação do banco (seção 5)
|
|   └─ frontend/
|       └─ index.html           # landing page (pública)
|       └─ cadastro.html
|       └─ login.html
|       └─ dashboard.html      # lista de roteiros do usuário
|       └─ roteiro.html        # detalhes/edição de um roteiro
|
|   específico
|       └─ perfil.html
|       └─ 404.html
|       └─ css/
|           └─ reset.css        # normalização básica
|           └─ variables.css    # cores, fontes, espaçamentos (custom
properties)
|               └─ style.css     # estilos gerais/compartilhados
|               └─ componentes.css # botões, cards, modais, formulários
|               └─ responsive.css # media queries (tablet/desktop)
|       └─ js/
|           └─ config.js        # URL base da API
|           └─ api.js           # funções genéricas de fetch (GET/POST)
|           └─ auth.js          # lógica de cadastro/login/logout
|           └─ dashboard.js     # lógica da tela de lista de roteiros
|           └─ roteiro.js       # lógica da tela de detalhes do roteiro
|           └─ perfil.js        # lógica da tela de perfil
|           └─ utils.js         # formatação de datas, moeda, validações
|       └─ assets/
|           └─ img/
|           └─ icons/
|
|   └─ README.md                # como instalar e rodar o projeto
```

Por que separar `backend/api` por "recurso"? Cada subpasta (`auth`, `roteiros`, `atividades`, `perfil`) representa uma entidade do sistema. Isso deixa a API fácil de navegar: quem procura "como editar um roteiro" sabe que o arquivo está em `backend/api/roteiros/editar.php`.

5. Banco de dados (MySQL)

5.1 Modelo conceitual

```
usuarios (1) ———< (N) roteiros (1) ———< (N) atividades
```

- Um **usuário** pode ter vários **roteiros**.
- Um **roteiro** pode ter várias **atividades**.
- Se um roteiro for excluído, suas atividades são excluídas junto (`ON DELETE CASCADE`).

5.2 Script de criação (`backend/database/schema.sql`)

```
sql
```

```
CREATE DATABASE IF NOT EXISTS roteiro_viagem
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE roteiro_viagem;

-- Tabela de usuários
CREATE TABLE usuarios (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    nome          VARCHAR(100) NOT NULL,
    email         VARCHAR(150) NOT NULL UNIQUE,
    senha_hash    VARCHAR(255) NOT NULL,
    criado_em     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Tabela de roteiros de viagem
CREATE TABLE roteiros (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id    INT NOT NULL,
    titulo        VARCHAR(150) NOT NULL,
    destino       VARCHAR(150) NOT NULL,
    data_inicio   DATE NOT NULL,
    data_fim      DATE NOT NULL,
    descricao     TEXT,
    imagem_capa   VARCHAR(255),
    criado_em     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Tabela de atividades dentro de um roteiro
CREATE TABLE atividades (
    id            INT AUTO_INCREMENT PRIMARY KEY,
```



```

roteiro_id      INT NOT NULL,
data_atividade  DATE NOT NULL,
horario         TIME,
titulo         VARCHAR(150) NOT NULL,
categoria       ENUM('hospedagem','transporte','alimentacao','passeio','d
local          VARCHAR(200),
custo_estimado  DECIMAL(10,2) DEFAULT 0,
observacoes     TEXT,
ordem          INT DEFAULT 0,
FOREIGN KEY (roteiro_id) REFERENCES roteiros(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE INDEX idx_rotteiros_usuario ON roteiros(usuario_id);
CREATE INDEX idx_atividadesroteiro ON atividades(roteiro_id, data_atividade)

```

5.3 Descrição dos campos

usuarios

Campo	Tipo	Observação
id	INT	chave primária
nome	VARCHAR(100)	nome exibido na interface
email	VARCHAR(150)	único, usado para login
senha_hash	VARCHAR(255)	gerado com <code>password_hash()</code> , nunca a senha em texto puro
criado_em	TIMESTAMP	data de criação da conta

roteiros

Campo	Tipo	Observação
id	INT	chave primária
usuario_id	INT	dono do roteiro (chave estrangeira)
titulo	VARCHAR(150)	ex.: "Férias em Lisboa"
destino	VARCHAR(150)	ex.: "Lisboa, Portugal"
data_inicio / data_fim	DATE	período da viagem
descricao	TEXT	texto livre, opcional
imagem_capa	VARCHAR(255)	caminho/URL da imagem (pode ficar vazio no MVP)

atividades

Campo	Tipo	Observação
id	INT	chave primária
roteiro_id	INT	roteiro ao qual pertence
data_atividade	DATE	em qual dia da viagem acontece
horario	TIME	opcional
titulo	VARCHAR(150)	ex.: "Visita ao Castelo de São Jorge"
categoria	ENUM	hospedagem / transporte / alimentacao / passeio / outro
local	VARCHAR(200)	endereço ou nome do lugar
custo_estimado	DECIMAL(10,2)	usado no cálculo de orçamento (ver seção 13)
ordem	INT	permite reordenar atividades manualmente dentro do mesmo dia

6. Fluxo de autenticação

1. **Cadastro:** o front-end envia (nome), (email), (senha) para (POST /api/auth/registrar.php). O back-end valida os campos, verifica se o e-mail já existe, gera o hash da senha com (password_hash()) e insere o usuário.
2. **Login:** o front-end envia (email) e (senha) para (POST /api/auth/login.php). O back-end busca o usuário pelo e-mail, compara a senha com (password_verify()) e, se correto, grava (\$_SESSION['usuario_id']).

3. **Verificação de sessão:** toda página protegida (dashboard, roteiro, perfil) chama `GET /api/auth/usuario_logado.php` ao carregar. Se não houver sessão válida, o JavaScript redireciona para `login.html`.
4. **Logout:** `POST /api/auth/logout.php` destrói a sessão (`session_destroy()`).
5. **Proteção da API:** todo endpoint que mexe em dados privados (`roteiros/*`, `atividades/*`, `perfil/*`) começa incluindo `auth_check.php`, que interrompe a execução com erro 401 se não houver usuário logado.

6.1 Exemplo — conexão com o banco (`backend/config/database.php`)

```
php

<?php
$host = 'localhost';
$dbname = 'roteiro_viagem';
$usuario = 'root';
$senha = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $usuario, $senha);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['erro' => 'Falha na conexão com o banco de dados']);
    exit;
}
```

6.2 Exemplo — verificação de sessão (`backend/includes/auth_check.php`)

```
php
```

```
<?php
session_start();
header('Content-Type: application/json');

if (!isset($_SESSION['usuario_id'])) {
    http_response_code(401);
    echo json_encode(['erro' => 'Você precisa estar logado para acessar este
    exit;
}
```

6.3 Exemplo completo — cadastro ((backend/api/auth/registrar.php))

Este arquivo serve como **modelo** para os demais endpoints: recebe JSON, valida, usa *prepared statements* e devolve uma resposta JSON padronizada.

php

```
<?php
session_start();
header('Content-Type: application/json');
require '../config/database.php';

$dados = json_decode(file_get_contents('php://input'), true);

$nome = trim($dados['nome'] ?? '');
$email = trim($dados['email'] ?? '');
$senha = $dados['senha'] ?? '';

if ($nome === '' || $email === '' || $senha === '') {
    http_response_code(400);
    echo json_encode(['erro' => 'Preencha nome, e-mail e senha.']);
    exit;
}

if (strlen($senha) < 6) {
    http_response_code(400);
    echo json_encode(['erro' => 'A senha deve ter pelo menos 6 caracteres.']);
    exit;
}

// verifica se o e-mail já está cadastrado
$stmt = $pdo->prepare('SELECT id FROM usuarios WHERE email = ?');
$stmt->execute([$email]);
if ($stmt->fetch()) {
    http_response_code(409);
    echo json_encode(['erro' => 'Este e-mail já está cadastrado.']);
    exit;
}
```

```
$senhaHash = password_hash($senha, PASSWORD_BCRYPT);

$stmt = $pdo->prepare('INSERT INTO usuarios (nome, email, senha_hash) VALUES
$stmt->execute([$nome, $email, $senhaHash]);

echo json_encode(['sucesso' => true, 'mensagem' => 'Conta criada com sucesso!']);
```

A partir daqui, os demais endpoints (`login.php`, `criar.php` de roteiros, etc.) seguem o **mesmo padrão**: ler JSON do corpo da requisição → validar → executar consulta com PDO usando `(?)` como placeholder → devolver JSON. A implementação completa de cada um fica como exercício, usando este arquivo como referência.

6.4 Nota sobre CORS (apenas se front e back rodarem em portas/domínios diferentes)

Se você usar a extensão "Live Server" para o front-end (ex.: `http://127.0.0.1:5500`) enquanto o PHP roda em outra porta (ex.: `http://localhost:8000`), adicione no início dos arquivos PHP (ou em `includes/cors.php`, incluído por todos):

```
php

header("Access-Control-Allow-Origin: http://127.0.0.1:5500");
header("Access-Control-Allow-Credentials: true");
header("Access-Control-Allow-Headers: Content-Type");
header("Access-Control-Allow-Methods: GET, POST, OPTIONS");
```

E, no `fetch()` do front-end, envie sempre `credentials: 'include'` para que o cookie de sessão seja enviado junto.

7. Telas do sistema (com textos)

Todas as telas usam tags semânticas (`<header>`, `<nav>`, `<main>`, `<section>`),

`<article>`, `<footer>`) e devem ser responsivas.

7.1 `index.html` — Página inicial (pública)

Objetivo: apresentar o produto e levar o visitante para cadastro/login.

Elementos:

- Cabeçalho com logo e navegação (`Entrar`, `Criar conta`)
- Seção principal (hero) com chamada para ação
- Seção com 3 cards de funcionalidades
- Rodapé

Textos sugeridos:

Logo: Roteirize

Título (hero): Planeje sua próxima viagem em minutos

Subtítulo: Organize destinos, atividades e horários em um só lugar, sem planilhas.

Botão principal: Começar agora — é grátis

Link secundário: Já tenho uma conta

Cards de funcionalidades:

1. "Roteiros personalizados" — Crie um roteiro para cada viagem, com destino e datas.
2. "Organizado por dia" — Distribua suas atividades dia a dia, na ordem que quiser.
3. "Acesse de qualquer lugar" — Seu roteiro fica salvo e disponível onde você estiver.

Rodapé: © 2026 Roteirize — Projeto acadêmico

7.2 `cadastro.html` — Criar conta

Elementos: formulário com nome, e-mail, senha e confirmação de senha.

Textos sugeridos:

Título: Criar conta

Rótulos dos campos:

- Nome completo
- E-mail
- Senha
- Confirmar senha

Botão: Cadastrar

Link: Já tem uma conta? Entrar

Mensagens de erro possíveis:

- "Preencha todos os campos."
- "As senhas não coincidem."
- "Este e-mail já está cadastrado."
- "A senha deve ter pelo menos 6 caracteres."

Mensagem de sucesso: "Conta criada com sucesso! Redirecionando para o login..."

7.3 `login.html` — Entrar

Textos sugeridos:

Título: Entrar

Rótulos dos campos:

- E-mail
- Senha

Botão: Entrar

Link: Ainda não tem conta? Cadastre-se

Mensagem de erro: "E-mail ou senha inválidos."

7.4 `dashboard.html` — Meus roteiros (área logada)

Objetivo: listar os roteiros do usuário e permitir criar novos.

Elementos:

- Cabeçalho com nome do usuário logado e botão "Sair"
- Botão "+ Novo roteiro" (abre um formulário/modal)
- Grade (grid) de cards, um por roteiro
- Estado vazio (quando não há roteiros ainda)

Textos sugeridos:

Cabeçalho: Olá, {nome do usuário} [Sair]

Título da página: Meus roteiros

Botão: + Novo roteiro

Card de roteiro:

- Título do roteiro
- Destino
- Datas (ex.: "10 a 18 de agosto de 2026")
- Botões: Ver detalhes · Editar · Excluir

Estado vazio:

"Você ainda não criou nenhum roteiro."

"Que tal planejar sua próxima viagem agora?"

Botão: Criar meu primeiro roteiro

Confirmação de exclusão:

"Tem certeza que deseja excluir este roteiro? Essa ação não pode ser desfeita."

Botões: Cancelar / Excluir

Formulário "Novo roteiro" / "Editar roteiro" (modal):

Título: Novo roteiro

Rótulos dos campos:

- Título do roteiro (ex.: Férias em Lisboa)
- Destino
- Data de início
- Data de término
- Descrição (opcional)

Botão: Salvar roteiro

7.5 `roteiro.html?id={id}` — Detalhes do roteiro

Objetivo: mostrar as atividades do roteiro organizadas por dia, permitindo adicionar/editar/excluir.

Elementos:

- Cabeçalho com título do roteiro, destino e datas; botão "Editar roteiro"
- Lista de dias da viagem, cada um com suas atividades em ordem cronológica
- Botão "+ Adicionar atividade" (por dia ou geral, com campo de data)
- Resumo do custo total estimado (soma de `custo_estimado`)

Textos sugeridos:

```
Cabeçalho: {Título do roteiro}
Subtítulo: {Destino} · {data_inicio} a {data_fim}
Botão: Editar roteiro

Para cada dia:
"Dia 1 – 10 de agosto"
  Lista de atividades: horário, título, categoria, local

Botão: + Adicionar atividade

Estado vazio (sem atividades):
"Nenhuma atividade cadastrada ainda. Comece adicionando a primeira!"

Rodapé de custos:
"Custo total estimado: R$ {soma}"
```

Formulário "Nova atividade" / "Editar atividade" (modal):

Título: Nova atividade

Rótulos dos campos:

- Data
- Horário (opcional)
- Título da atividade (ex.: Visita ao Castelo de São Jorge)
- Categoria (select: Hospedagem / Transporte / Alimentação / Passeio / Outro)
- Local
- Custo estimado (opcional)
- Observações (opcional)

Botão: Salvar atividade

7.6 `perfil.html` — Meu perfil

Textos sugeridos:

Título: Meu perfil

Rótulos:

- Nome
- E-mail (somente leitura)
- Nova senha (opcional – deixe em branco para não alterar)

Botão: Salvar alterações

Mensagem de sucesso: "Dados atualizados com sucesso!"

7.7 `404.html` — Página não encontrada

Título: 404 – Página não encontrada

Texto: A página que você procura não existe ou foi movida.

Botão: Voltar para a página inicial

8. Especificação da API (back-end PHP)

Todas as respostas são JSON. Endpoints marcados como "Sim" em **Login?** exigem sessão ativa (retornam `401` caso contrário).

Autenticação

Método	Endpoint	Login?	Corpo/Parâmetros	Resposta
POST	<code>/api/auth/registrar.php</code>	Não	<code>nome, email, senha</code>	<code>{sucesso, mensagem}</code>
POST	<code>/api/auth/login.php</code>	Não	<code>email, senha</code>	<code>{sucesso, usuario: {id,nome, email}}</code>
POST	<code>/api/auth/logout.php</code>	Sim	—	<code>{sucesso}</code>
GET	<code>/api/auth/usuario_logado.php</code>	Sim	—	<code>{usuario: {id,nome, email}}</code>

Roteiros

Método	Endpoint	Login?	Corpo/Parâmetros	Resposta
GET	<code>/api/roteiros/listar.php</code>	Sim	—	<code>{roteiros: ...}</code>

Método	Endpoint	Login?	Corpo/Parâmetros	Resposta
				[...]}
GET	/api/roteiros/detalhes.php?id=	Sim	query id	{roteiro: ..., atividades: [...]}
POST	/api/roteiros/criar.php	Sim	titulo, destino, data_inicio, data_fim, descricao	{sucesso, id}
POST	/api/roteiros/editar.php	Sim	id, titulo, destino, data_inicio, data_fim, descricao	{sucesso}
POST	/api/roteiros/excluir.php	Sim	id	{sucesso}

Atividades

Método	Endpoint	Login?	Corpo/Parâmetros	Resposta
GET	/api/atividades/listar.php?roteiro_id=	Sim	query roteiro_id	{atividades: [...]}
POST	/api/atividades/criar.php	Sim	roteiro_id, data_atividade,	{sucesso, id}

Método	Endpoint	Login?	Corpo/Parâmetros	Resposta
			horario, titulo, categoria, local, custo_estimado, observacoes	
POST	/api/atividades/editar.php	Sim	id, ...mesmos campos	{sucesso}
POST	/api/atividades/excluir.php	Sim	id	{sucesso}

Perfil

Método	Endpoint	Login?	Corpo/Parâmetros	Resposta
POST	/api/perfil/atualizar.php	Sim	nome, nova_senha (opcional)	{sucesso}

Por que usar POST em vez de PUT/DELETE? Para simplificar: navegadores lidam melhor com GET/POST em formulários e alguns servidores compartilhados restringem outros métodos. Em cada endpoint POST de edição/exclusão, o próprio back-end verifica se o `usuario_id` da sessão é o dono do registro antes de alterar/apagar — isso impede que um usuário edite dados de outro (RF13).

9. Organização do front-end (CSS e JS)

CSS

Arquivo	Conteúdo
<code>reset.css</code>	zera margens/paddings padrão do navegador
<code>variables.css</code>	<code>:root { --cor-primaria: ...; --espacamento-md: ...; }</code>
<code>style.css</code>	tipografia, layout base, header/footer
<code>componentes.css</code>	botões, cards, formulários, modais, badges de categoria
<code>responsive.css</code>	media queries (ver seção 10)

JavaScript

Arquivo	Responsabilidade
<code>config.js</code>	define <code>const API_BASE = 'http://localhost/roteirize/backend/api';</code>
<code>api.js</code>	funções genéricas <code>apiGet(caminho)</code> e <code>apiPost(caminho, dados)</code> que já tratam <code>fetch</code> , <code>JSON.stringify</code> , cabeçalhos e erros
<code>auth.js</code>	eventos de submit dos formulários de cadastro/login/logout; redireciona conforme sessão
<code>dashboard.js</code>	busca e renderiza os cards de roteiros; abre modal de criação/edição

Arquivo	Responsabilidade
<code>roteiro.js</code>	busca detalhes do roteiro, agrupa atividades por dia, renderiza a timeline, controla o modal de atividade
<code>perfil.js</code>	carrega e salva dados do perfil
<code>utils.js</code>	<code>formatarData()</code> , <code>formatarMoeda()</code> , validação simples de e-mail

Exemplo de `api.js` (esqueleto):

javascript

```
async function apiPost(caminho, dados) {
  const resposta = await fetch(`${API_BASE}${caminho}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify(dados),
  });
  const json = await resposta.json();
  if (!resposta.ok) throw new Error(json.erro || 'Erro inesperado');
  return json;
}

async function apiGet(caminho) {
  const resposta = await fetch(`${API_BASE}${caminho}`, { credentials: 'include' });
  const json = await resposta.json();
  if (!resposta.ok) throw new Error(json.erro || 'Erro inesperado');
  return json;
}
```

10. Responsividade

Adote a abordagem **mobile-first**: escreva primeiro o CSS para celular e depois "abra" o layout para telas maiores com `(min-width)`.

Faixa	Largura	Layout sugerido
Celular	até 599px	1 coluna, menu simplificado, cards empilhados
Tablet	600px a 1023px	2 colunas de cards, menu horizontal
Desktop	a partir de 1024px	3+ colunas de cards, sidebar opcional na tela de roteiro

CSS

```
/* base: celular */
.grid-roteiros { display: grid; grid-template-columns: 1fr; gap: 1rem; }

/* tablet */
@media (min-width: 600px) {
  .grid-roteiros { grid-template-columns: repeat(2, 1fr); }
}

/* desktop */
@media (min-width: 1024px) {
  .grid-roteiros { grid-template-columns: repeat(3, 1fr); }
}
```

Teste sempre no DevTools do navegador (modo responsivo) nas três larguras acima.

11. Segurança — boas práticas mínimas

- **Nunca** salvar senha em texto puro — sempre `password_hash()` / `password_verify()`.
 - **Sempre** usar *prepared statements* do PDO (`(?)` ou `:nome`) — nunca concatenar valores direto na query SQL.
 - Validar os dados também no back-end, mesmo que o front-end já valide (o usuário pode burlar o JavaScript).
 - Ao exibir dados vindos do usuário em HTML, usar `htmlspecialchars()` para evitar XSS.
 - Checar em cada endpoint de edição/exclusão se o registro pertence ao usuário da sessão antes de alterar.
 - Não expor mensagens de erro internas do banco (`($e->getMessage())`) diretamente ao usuário final — logar no servidor e devolver uma mensagem genérica.
-

12. Roteiro sugerido de implementação

1. Instalar ambiente local (XAMPP/MAMP/Laragon) e criar o banco com `schema.sql`.
2. Criar a estrutura de pastas da seção 4 (mesmo vazia).
3. Implementar `config/database.php` e testar a conexão.
4. Implementar o fluxo de autenticação no back-end (`registrar.php`, `login.php`, `logout.php`, `usuario_logado.php`, `auth_check.php`).
5. Construir `cadastro.html` e `login.html`, integrando com `auth.js`.
6. Implementar o CRUD de roteiros no back-end.
7. Construir `dashboard.html`, listando e criando roteiros (`dashboard.js`).
8. Implementar o CRUD de atividades no back-end.

9. Construir `roteiro.html`, exibindo atividades agrupadas por dia (`roteiro.js`).
 10. Construir `perfil.html` e o endpoint de atualização de perfil.
 11. Aplicar o CSS responsivo (mobile-first) em todas as telas.
 12. Testar tudo com o checklist da seção 14 e corrigir bugs.
 13. (Opcional) Implementar melhorias da seção 13.
-

13. Funcionalidades extras (opcional)

Ideias para quem terminar o MVP e quiser enriquecer o projeto:

- **Upload de imagem de capa** do roteiro (`move_uploaded_file`).
 - **Compartilhamento** do roteiro por link público, somente leitura.
 - **Exportar roteiro em PDF**.
 - **Mapa** com os locais das atividades (biblioteca Leaflet.js, gratuita).
 - **Modo escuro** (dark mode) usando variáveis CSS.
 - **Busca/filtro** de roteiros por destino ou data.
 - **Gráfico de orçamento** por categoria (ex.: biblioteca Chart.js).
 - **Reordenar atividades** por arrastar-e-soltar (drag and drop nativo do HTML5).
-

14. Checklist de testes manuais

- ☐ Não é possível se cadastrar com e-mail já existente
- ☐ Não é possível fazer login com senha errada
- ☐ Usuário deslogado é redirecionado ao tentar acessar `dashboard.html` ou `roteiro.html` diretamente

- ☐ Roteiro criado aparece imediatamente no dashboard
- ☐ Editar um roteiro reflete a mudança na lista e nos detalhes
- ☐ Excluir um roteiro também remove suas atividades
- ☐ Atividades aparecem agrupadas corretamente por dia e em ordem cronológica
- ☐ Usuário A não consegue ver/editar/excluir roteiros do usuário B (testar trocando o `id` na URL)
- ☐ Layout funciona sem quebrar em 360px (celular), 768px (tablet) e 1280px (desktop)
- ☐ Mensagens de erro do formulário aparecem de forma clara para o usuário